Grouped-Query Attention and Partial RoPE Under a Fixed FLOP Budget: A Controlled Study of a Small Language Model Trained from Scratch

Jerry Yang Shao Ern
A0281549U · e1155677@u.nus.edu

Github Repository: https://github.com/mcxraider/modded-nanochat (experiment namespace `d8-v5`; code commit `c07aa7e`)

Abstract

I test grouped-query attention and partial rotary embeddings in 14 from-scratch depth-8 language-model runs under a fixed estimated training-FLOP allowance. Multi-head attention achieves mean held-out test BPB 0.960795; two K/V heads and one K/V head worsen it by 0.005158 and 0.011370 despite longer training. Training throughput improves by 2.84% and 6.41%, and allocated K/V cache halves and quarters, respectively. Extra tokens recover about one quarter of same-seed matched-token quality penalties; the value-embedding interaction remains unresolved. Partial RoPE has no resolved quality effect or measured training speedup. Eager H100 decoding does not exhibit the predicted GQA speedup. The results distinguish arithmetic savings, memory savings and measured speed, while retaining failed predictions and uncertainty from limited seeds.

Compute budget (declared before any training run). Hardware: one NVIDIA H100 SXM 80 GB on Modal, BF16 autocast, FlashAttention-3 [SBZ+24]. Per-run allowance: at most $C = 1.385 \times 10^{17}$ *estimated* training FLOPs, the cost of the baseline B0 trained on upstream nanochat's compute-optimal horizon at depth 8 (1,680 updates $\times$ 262,144 tokens $\times$ 314,575,872 FLOPs/token). FLOPs are counted with nanochat's estimator (6 FLOPs per matmul parameter per token plus windowed attention), so each architecture receives the largest whole number of updates whose estimate fits in C. A 2-hour wall-clock cap applies to every run as a safety stop only; no comparable run reached it (each needed about 6.77–7.61 minutes). Pilots, numerical preflight checks and tokenizer training are excluded from the budget and reported separately.

Initialisation. Every model is trained from random initialisation. No pretrained weights, embeddings or other pretrained neural components are used. I reuse the nanochat architecture and code, its BPE tokenizer recipe (trained by me on the training shards, vocabulary 32,768) and the ClimbMix corpus.

Evaluation objective. Bits per byte (BPB) of next-token prediction on held-out ClimbMix web text: 2^{24} tokens from shard 06542 (validation, used for curves and any decision) and, separately, 2^{24} tokens from shard 06541 (test, evaluated once per run after training; the primary metric). BPB measures in-distribution language modelling, not downstream reasoning, instruction following or long-context quality; inference speed and memory are separate metrics.

1 Introduction and research question

Grouped-query attention (GQA) shares key/value (K/V) heads across query heads, reducing cache memory and projection FLOPs [ALTdJ+23, Sha19]. Partial rotary embeddings rotate only a fraction of each head's dimensions [SLP+21, BBH+22]. Their effects on small models trained from scratch under limited compute are less clear than their large-model motivations. At fixed compute, GQA buys extra training tokens but may lose per-token quality. I ask:

> *At depth 8 and a fixed budget of C estimated training FLOPs, what do GQA and partial RoPE cost or buy in held-out BPB, training throughput and inference memory/speed, and how much does GQA benefit from extra tokens?*

I pre-registered hypotheses and analysis rules in `experiments/experiments.md`. The study combines replicated MHA/GQA-2 comparisons, a rotary dose–response, value-embedding and matched-token controls, and H100 inference measurements.

2 Background

Multi-query and grouped-query attention. MHA [VSP+17] gives each query head independent K/V projections. MQA shares one K/V head; GQA uses an intermediate number. This reduces cache reads, which can dominate decoding at large batch or long context [Sha19, PDC+23]. GQA's near-MHA quality evidence comes from uptrained T5-XXL checkpoints [ALTdJ+23], motivating my from-scratch test. Training saves only K/V projection matmuls: attention products still use every query head. Here GQA-2 saves 4.0% of estimated FLOPs and MQA 6.0%.

Rotary embeddings and partial RoPE. RoPE [SLP+21] rotates query/key dimension pairs by position-dependent angles, with frequencies $\theta_i = 10000^{-2i/d}$. GPT-NeoX ro-

tates only 25% of each head [BBH+22]; evidence that low-frequency pairs serve almost position-independent semantic roles also motivates partial rotation [BVP+25]. My implementation recomputes frequencies over the rotated prefix: R25 uses 16 rather than 64 pairs, with lowest frequency $10^{-3.75}$ rather than $10^{-3.9375}$, leaving 96 dimensions unrotated. Thus the fraction changes both positional capacity and the frequency grid.

Value embeddings. nanochat's attention adds a learned, token-indexed *value embedding* to the values in alternating layers, scaled by an input-dependent gate. This is a form of ResFormer value-residual learning [ZWJ+24], which reports the same loss with 16% fewer parameters. The embedding width equals the value width, $n_{\text{kv}} \cdot d_{\text{head}}$. At depth 8 these tables hold 67.1M of B0's 125.8M parameters, so halving the K/V heads removes 33.5M embedding parameters, about 16 times the 2.1M removed from the K/V projections. Nanochat's FLOP estimator treats embedding lookups as free, so this change is invisible to the budget. A naive "GQA vs MHA" comparison in this codebase is therefore also a "half vs full value embeddings" comparison, and Experiment E5 tests whether their effects interact.

Compute-optimal training. Chinchilla [HBM+22] models loss as $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$; embedding parameters require separate treatment [KMH+20]. I adopt nanochat's fitted horizon of 10.5 tokens per scaling parameter (transformer matrices plus unembedding) [Kar25], using Chinchilla's data term only to predict the approximate benefit of GQA's extra tokens (§3.3).

3 Methods and experimental setup

3.1 Model, data and training procedure

Baseline B0. B0 is the nanochat ClimbMix leaderboard model at upstream commit `324e69c` (the configuration immediately before the "autoresearch" rounds), rescaled to depth 8: 8 layers, width 512, 4 heads of dimension 128, context 2,048, vocabulary 32,768. It keeps that commit's design: pre-norm with parameter-free RMSNorm, QK-norm [HDPC20], squared-ReLU MLP [SML+21] of width $4\times$, untied embeddings, logit soft-capping, learnable residual and input-skip scalars, gated value embeddings in alternating layers, and sliding-window attention with the pattern SSSL (short windows of 1,024 tokens, every fourth and the final layer full) [BPC20]. I made one deliberate change: I train in BF16 rather than the leaderboard's FP8, so that numerical precision cannot interact with the head shapes.

Variants. I extended the model with three switches, each verified by CPU unit tests and a GPU preflight (§3.5): the number of K/V heads $n_{\text{kv}} \in \{4, 2, 1\}$; the rotary fraction $f \in \{1, 0.75, 0.25\}$, where the first $f \cdot d_{\text{head}}$ dimensions of q and k are rotated and the rest pass through unchanged; and the MLP ratio and value-embedding on/off flags used by the controls. Table 1 lists the eight resulting architectures.

Data and tokenizer. I use NVIDIA ClimbMix [DYF+25], a 400B-token web mixture, in nanochat's shard format. Training uses shards 0–47; 06542 is validation and 06541 is test, never used for decisions. My 2^{15}-vocabulary BPE trainer reads training files in order up to 2×10^9 characters, excluding held-out shards by recipe; its retained export does not enumerate every input. BOS-aligned best-fit packing produces 2,048-token rows. All runs end in the first epoch, with seed rotations 0, 16 and 32 over 48 shards. The pool's exact token capacity was not measured.

Optimisation. Following the pinned upstream training script, matrices are updated with Muon [JJB+24] (Polar-Express orthogonalisation [APMG25] and NorMuon-style factored second moments [LLL+25]) and the embeddings, unembedding and scalars with AdamW. Upstream's own depth-8 rules set the global batch to 262,144 tokens, all learning rates scaled by $\sqrt{262144/524288} = 0.707$, and the Muon weight decay to 0.371, computed once from B0 and *frozen for every variant.* The schedule has no warm-up, a constant learning rate for 50% of updates and then linear decay to zero. Weight decay decays linearly with progress, and Muon momentum warms up from 0.85 to 0.95 over 300 updates. I tuned no hyperparameters for any variant (see Limitations, §5).

Metric definition. For held-out target tokens x_i whose decoded UTF-8 byte lengths are b_i, the evaluation computes

$$\text{BPB} = \frac{\sum_i -\log p(x_i \mid x_{<i})}{\log(2) \sum_i b_i}.$$

Lower is better. Comparisons share tokenizer, held-out files and masking. Each final split requests 2^{24} tokens; saved scores and configurations agree, but independent consumed-token counters were not retained.

3.2 The FLOP budget and why I chose it

Estimated FLOPs provide a reproducible arithmetic budget and reward GQA for removed projections. The estimator omits non-matmul work, including embedding lookups and rotary operations; measured throughput is therefore reported separately (§4.4). Training refuses targets above C and stops at the assigned token count. Every fixed-compute run uses at least 99.98% of C.

3.3 Experimental design

I ran 14 training runs. Apart from the stated change, all share data, tokenizer, context, batch, optimiser, schedule and evaluation text, and all variants use seed 1 unless noted. The seed sets both the initialisation and the training-shard rotation, so seed noise includes data-order noise.

- **E1, noise floor** (`d8-b0-s1..s3`): B0 with seeds 1–3. The sample s.d. s of test BPB defines the resolution threshold.
- **E2, main comparison** (`d8-gqa2-s1..s3, -r75`): GQA-2 (seeds 1–3) against the B0 mean, and GQA-2-R75 against both. The three GQA-2 seeds also check that the noise is similar across architectures.
- **E3, parameter-matched control** (`d8-gqa2-pm`): GQA-2-PM spends GQA's saving on a wider MLP (2,304 instead of 2,048) instead of extra tokens. It matches B0's

Table 1: The eight architectures (depth 8, width 512, 4 query heads of dimension 128). FLOPs/token is nanochat's estimate. "Updates" is the largest whole number of 262,144-token updates within C. Scaling parameters are the transformer matrices plus the unembedding, and total parameters add the token and value embeddings. KV cache is in bytes per token (BF16, all layers).

Name	Change from B0	FLOPs/token	Updates	Tokens	Scaling/total params	KV B/tok.
B0	— (MHA, $n_{\text{kv}} = 4$)	314.58M	1,680	0.440B	41.94M / 125.83M	16,384
GQA-2	$n_{\text{kv}} = 2$	301.99M	1,750	0.459B	39.85M / 90.18M	8,192
GQA-2-R75	GQA-2, 75% RoPE	301.99M	1,750	0.459B	39.85M / 90.18M	8,192
GQA-2-R25	GQA-2, 25% RoPE	301.99M	1,750	0.459B	39.85M / 90.18M	8,192
GQA-2-PM	GQA-2, MLP 4.5×	314.57M	1,680	0.440B	41.94M / 92.27M	8,192
MQA (GQA-4)	$n_{\text{kv}} = 1$	295.70M	1,787	0.468B	38.80M / 72.35M	4,096
B0-noVE	B0, no value emb.	314.57M	1,680	0.440B	41.94M / 58.72M	16,384
GQA-2-noVE	GQA-2, no value emb.	301.99M	1,750	0.459B	39.85M / 56.62M	8,192

scaling parameters to 256 parameters and its FLOPs to 0.0005%.

- **E4, dose–response** (`d8-gqa4`, `d8-gqa2-r25`): MQA (K/V axis, against B0 and GQA-2) and GQA-2-R25 (rotary axis, against GQA-2 and R75).
- **E5, value-embedding** 2×2 (`d8-b0-nove`, `d8-gqa2-nove`): B0-noVE and GQA-2-noVE. With B0 and GQA-2 these cross $\{4, 2\}$ K/V heads with value embeddings $\{\text{on}, \text{off}\}$.
- **E6, matched-token ablation** (`d8-gqa2-tok`, `d8-gqa4-tok`): GQA-2 and MQA trained on B0's 1,680 updates (96% and 94% of C). The difference to their fixed-compute twin (same seed, so the same initial weights and batches up to update 1,680) estimates the benefit of the longer token horizon and its associated schedule; the difference to B0 is the matched-token cost of the complete architectural change, including value embeddings.

Pre-registered predictions. Table 5 lists every registered prediction alongside its outcome. For H6, the transferred Chinchilla data term ($B = 410.7$, $\beta = 0.28$, $B/D^{\beta} \approx 1.56$ nats at $D = 0.44$B) predicts that 4.2% / 6.4% extra tokens save $0.018/0.027$ nats, or roughly 0.005–0.006 / 0.008–0.009 BPB at 4.2–4.8 bytes per token.

Statistical procedure. A difference counts as *resolved* only if it exceeds $2\sqrt{2}\,s$, a two-sigma bound on the difference of two single runs, where s is the s.d. of the three B0 seeds. This rule is crude: s has two degrees of freedom. By simulation it flags about 18% of null single-vs-single contrasts, so across my ≈ 17 contrasts two or three spurious "resolved" flags can occur [BDB+21]. I therefore pre-registered three secondary analyses and report where they disagree with the primary rule: (i) a bound that accounts for seed means, $2s\sqrt{1/n_1 + 1/n_2}$; (ii) a Student-t 95% interval using s_p, the s.d. pooled over the B0 and GQA-2 groups (df $= 4$); and (iii) a *paired* analysis that uses the shared data order of same-seed runs (E2 seed pairs and E6 twins). Validation curves are compared only between runs with the same token target, because a longer run's learning-rate decay starts later.

3.4 Inference benchmark

After training, each final checkpoint is benchmarked on the H100 with fixed random prompts, greedy decoding of exactly 256 tokens, a batch size $\in \{1, 4, 16, 64\}$ and a prompt length $\in \{128, 1792, 8192\}$. A 1,792-token prompt plus 256 generated tokens exactly fills the 2,048 training context. The 8,192-token prompt exceeds the context, so I use it only to measure speed and memory, with no quality claim. I report prefill throughput, time to first token, decode throughput (the remaining 255 tokens), peak memory and the allocated KV cache, and compare them with the analytic cache size and the K/V bytes read per decoded token. Execution is eager. Weights are cast to BF16 once, and the cache position is kept on the host to avoid a GPU synchronisation per step.

3.5 Validity checks before the main runs

CPU tests verify full-RoPE parity, unchanged unrotated dimensions, GQA projection/cache shapes, cached/full-sequence decoding parity and bit-identical checkpoint resumption. All eight architectures passed H100 FA3/SDPA and compiled/eager checks across the sliding-window boundary; maximum recorded relative matrix-gradient error was 2.997% (5% tolerance, separate scalar tolerance). Ten-update pilots checked FA3 selection, inference and loading. Full runs had no nonfinite logged metrics, 168/168 successful inference cells and median loader waits at most 0.556% (2% gate). Pilot evidence is retained under `results/d8-v5/pilots/9a34115f44463cc7/`.

4 Results

4.1 Completion, noise floor and learning behaviour (E1)

All 14 runs reached their token targets: 24,047 contiguous updates and 6,303,776,768 training tokens in total, counting each model separately. No run stopped at the wall-clock cap or the spending gate. The 12 fixed-compute runs used 99.9862–100% of their allowance; the two E6 controls deliberately used 96% and 94%. Thus the comparisons below are completed experiments rather than extrapolations from the pilots.

B0's three test scores are 0.961227, 0.961118 and 0.960040 BPB (mean 0.960795), giving $s = 0.000656$ and the primary threshold $\tau = 2\sqrt{2}s = 0.001855$ BPB. The predicted noise range $[0.001, 0.003]$ was too high for B0. GQA-2's scores are 0.967636, 0.966122 and 0.964101 BPB (mean 0.965953), giving $s_{\text{GQA}-2} = 0.001773$, 2.70 times B0's s.d. The pooled estimate is $s_p = 0.001337$ (4 degrees of freedom). Consequently, a threshold derived only from B0 is optimistic for GQA comparisons; the sensitivity intervals materially affect

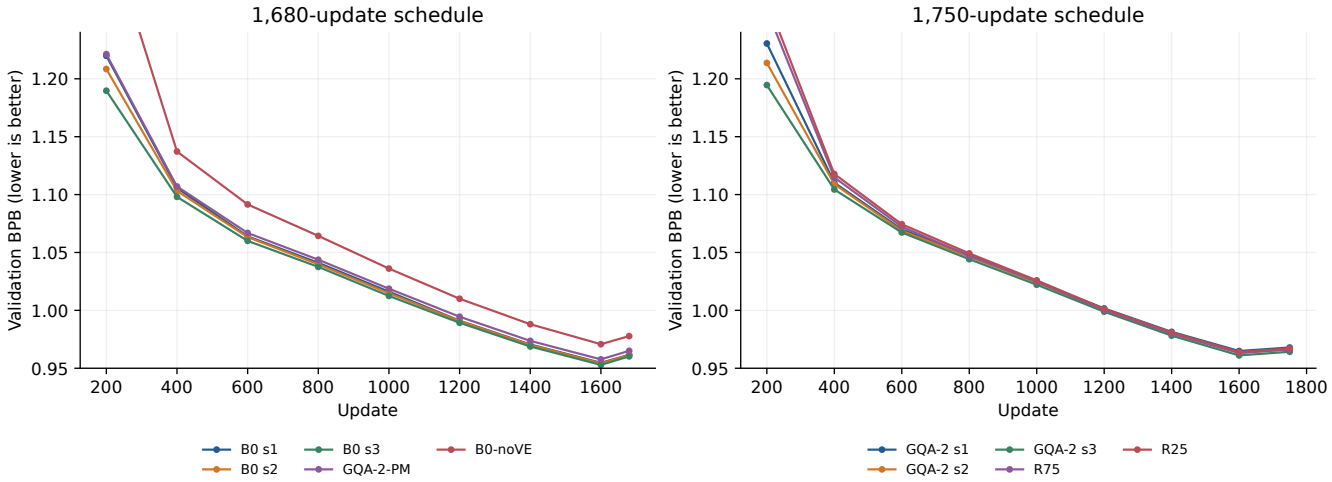


Figure 1: Recorded validation curves, grouped by the 1,680- and 1,750-update schedules. Final validation is included. The right panel compares full, 75% and 25% RoPE at the same horizon. Legends are outside the plotted data.

Table 2: Final quality and steady training throughput. B0 and GQA-2 are three-seed means; all other entries have one seed. Δ is test BPB relative to the stated reference. P/M/T denote resolution under the primary, seed-mean and pooled-t 95% rules (Y/N). These are comparison rules, not equivalence tests. Tok/s ratios use the B0 three-seed mean.

Model	Test BPB	Val BPB	Reference	Δ test	P/M/T	Tok/s ratio
B0	0.960795	0.961092	—	—	—	1.000
GQA-2	0.965953	0.966193	B0	+0.005158	Y/Y/Y	1.028
GQA-2-R75	0.965797	0.966148	GQA-2	-0.000156	N/N/N	1.027
GQA-2-R25	0.966821	0.967022	GQA-2	+0.000868	N/N/N	1.024
GQA-2-PM	0.964867	0.965068	B0	+0.004072	Y/Y/N	0.991
MQA	0.972165	0.972318	B0	+0.011370	Y/Y/Y	1.064
B0-noVE	0.977541	0.977761	B0	+0.016746	Y/Y/Y	1.023
GQA-2-noVE	0.982192	0.982504	B0-noVE	+0.004651	Y/Y/N	1.042
GQA-2-tok	0.969950	0.970276	B0	+0.009155	Y/Y/Y	1.029
MQA-tok	0.975687	0.975845	B0	+0.014892	Y/Y/Y	1.062

the interpretation of several smaller effects.

Validation BPB exceeds test BPB by 0.000142–0.000351; the main GQA-2 penalty agrees across splits (0.005101 validation, 0.005158 test). Figure 1 shows continued improvement through decay. Curves use validation only and separate horizons because their schedules differ; test did not select checkpoints.

4.2 Quality at fixed compute (E2–E5)

Figure 2 summarises every final score and contrast; the paragraphs below discuss each experiment.

GQA-2 versus MHA (H2a). GQA-2 worsens test BPB by 0.005158 (0.537% relative) despite processing 4.17% more tokens and having 28.3% fewer total parameters. The pooled 95% interval is $[+0.002128, +0.008188]$. Every same-seed difference is positive ($+0.006409$, $+0.005004$, $+0.004061$; Figure 3, left); their mean has paired 95% interval $[+0.002223, +0.008093]$. H2a is contradicted by the primary rule and both sensitivity analyses. This is strong evidence for a quality penalty in this configuration, rather than a conclusion resting on one unlucky run.

The paired s.d. $s_d = 0.001182$ exceeds B0's equal-noise prediction $\sqrt{2}s$, but is below the independent heteroscedastic estimate $\sqrt{s_{\text{B0}}^2 + s_{\text{GQA}-2}^2} = 0.001891$. With only three pairs, the result identifies a penalty for the complete modification, including narrower value embeddings and longer training.

Stronger K/V sharing (H4a). The fixed-compute ordering is monotone (Figure 4, left): MHA 0.960795, GQA-2 0.965953, MQA 0.972165 BPB. MQA loses 0.011370 BPB against B0 and 0.006212 against GQA-2. The pooled intervals are $[+0.007085, +0.015656]$ and $[+0.001927, +0.010497]$, respectively, supporting H4a. Its 6.37% extra tokens do not recover the lost quality. Sharing K/V is a plausible capacity bottleneck because queries must use fewer independent key/value representations, but the experiment does not isolate this mechanism: MQA also quarters B0's value-embedding width and changes the gate and optimiser matrix shapes. The monotone association is established; a universal causal penalty for GQA is not.

Partial RoPE (H2c, H4b). At the shared 1,750-update horizon (Figure 4, centre), R75 changes test BPB by -0.000156 and R25 by $+0.000868$ against the GQA-2 mean. R25 is 0.001024 worse than R75. All three differences are unresolved; the pooled intervals against full RoPE are $[-0.004441, +0.004130]$ for R75 and

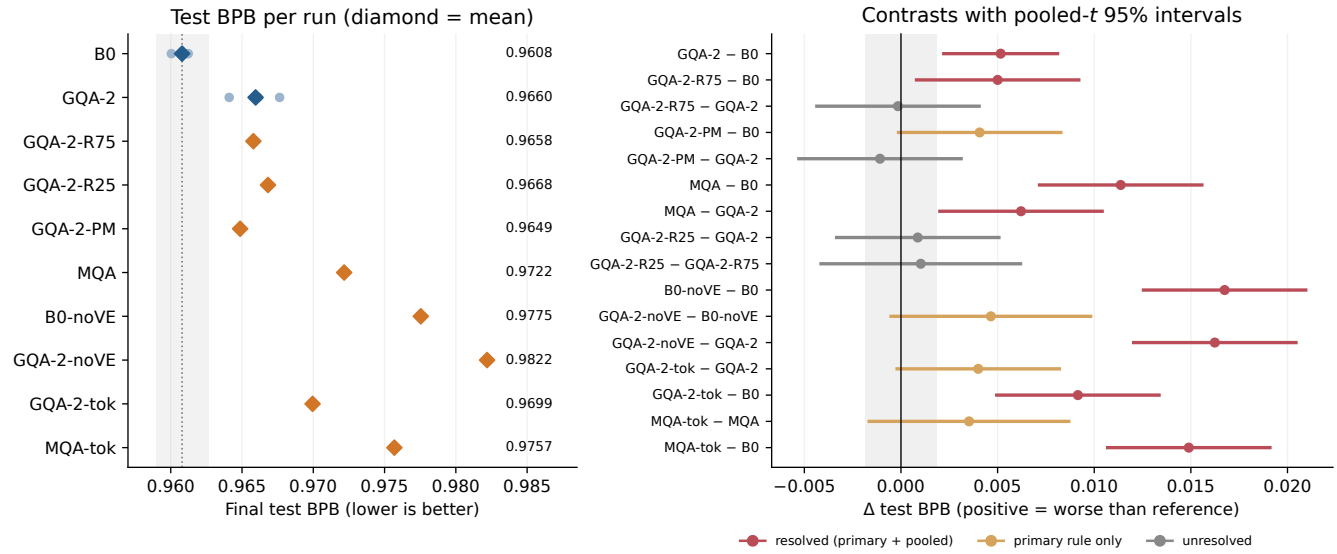


Figure 2: Quality summary. Left: final test BPB of every run (light dots) and each architecture's mean (diamonds, with values); the dotted line and grey band mark the B0 mean $\pm\tau$. Right: all 16 registered contrasts, showing the point estimate and pooled-t 95% interval. The grey band is the primary threshold $\pm\tau$. Red contrasts resolve under both the primary and pooled rules, orange under the primary rule only, and grey under neither.

$[-0.003417, +0.005154]$ for R25. The registered unresolved-quality predictions hold, but these wide intervals cannot establish equivalence. For a direct same-seed comparison, R75 and R25 improve on full-RoPE GQA-2 s1 by 0.001839 and 0.000815; this different point ordering illustrates why a favourable single seed should not be presented as a robust improvement.

These results neither establish a semantic-channel mechanism nor test context extrapolation; the ablation also changes the frequency grid.

Value embeddings (E5, H5a–c). Removing value embeddings costs B0 0.016746 BPB, supporting H5a, and costs GQA-2 0.016239 BPB. Both effects survive pooled sensitivity ($[+0.012461, +0.021032]$ and $[+0.011954, +0.020525]$). These are substantially larger than the GQA penalty itself, demonstrating the importance of retaining this component at the tested budget. The FLOP estimator barely charges it, making the large quality contribution especially relevant to this particular compute accounting.

The GQA gap shrinks only from 0.005158 with value embeddings to 0.004651 without them (Figure 3, right). The interaction is $+0.000507$, well below its primary bound 0.002142, and its pooled interval is $[-0.005553, +0.006567]$. Its point-estimate sign agrees with H5b and its unresolved status agrees with H5c, but there is no resolved evidence that the effect differs between the two settings. In particular, the data do not establish that shrinking value embeddings explains GQA's loss. The noVE GQA penalty itself resolves under the primary rule but not the pooled interval ($[-0.000597, +0.009899]$). More disabled-embedding seeds and a fixed-width embedding control are needed to separate these mechanisms.

Spending the saving on MLP width (E3, H3). PM reaches 0.964867 BPB: 0.004072 worse than B0 and 0.001086 better than GQA-2's mean. Thus matching scaling parameters and estimated FLOPs does not meet H3 under the primary rule. However, the PM–B0 pooled interval $[-0.000213, +0.008357]$ crosses zero; that particular loss is less secure than the main GQA result. The PM–GQA-2 interval also crosses zero ($[-0.005371, +0.003199]$), so the experiment cannot rank wider MLPs against extra tokens confidently. PM uses B0's 1,680 updates, but still lacks 33.5M of B0's value-embedding parameters. It is a match on scaling parameters, not total model capacity. Its altered Muon learning-rate scaling is an additional reason not to interpret it as a clean test of parameter location alone.

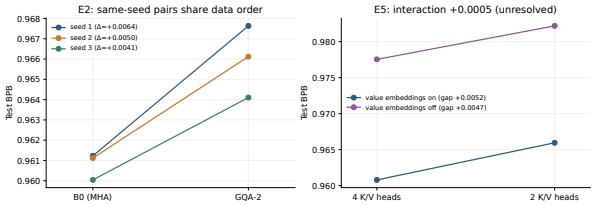


Figure 3: Left: E2 same-seed pairs; each line connects B0 and GQA-2 trained with the same seed, and hence the same data order. All three slopes are positive. Right: the E5 2×2 design (B0 and GQA-2 use three-seed means; noVE cells use one seed). The near-parallel lines show that removing value embeddings barely changes the GQA gap.

4.3 What the extra tokens buy (E6, H6)

The correct paired E6 comparisons use GQA-2 *seed 1*, not its three-seed mean (Figure 4, right):

$$\begin{aligned} \text{GQA-2:} \quad & 0.969950 \longrightarrow 0.967636 \quad (\Delta = -0.002314), \\ \text{MQA:} \quad & 0.975687 \longrightarrow 0.972165 \quad (\Delta = -0.003522). \end{aligned}$$

These gains are smaller than the predicted 0.005–0.006 and 0.008–0.009 BPB, respectively: extra data helps in the predicted direction, but the transferred scaling-law coefficients

Table 3: Full-run training performance. Throughput is each run's steady median, averaged across seeds for B0/GQA-2. TFLOP/s uses the same estimated FLOP model as the budget; it is not a hardware counter measurement. Memory is peak allocated memory in decimal GB (maximum across seeds), not reserved memory or total node usage.

Architecture	M tokens/s	vs. B0	Est. TFLOP/s	Train GB	Eval GB
B0	1.184	1.000	372.6	17.07	27.74
GQA-2	1.218	1.028	367.8	15.67	27.33
GQA-2-R75	1.216	1.027	367.3	15.66	27.33
GQA-2-R25	1.213	1.024	366.2	15.66	27.33
GQA-2-PM	1.174	0.991	369.3	16.26	27.37
MQA	1.260	1.064	372.7	14.97	27.12
B0-noVE	1.212	1.023	381.2	15.99	27.00
GQA-2-noVE	1.234	1.042	372.8	15.13	26.96

overpredict the measured benefit. The GQA-2 gain narrowly fails the secondary paired bound $2s_d = 0.002363$; MQA passes it. Neither gain resolves under the pooled unpaired sensitivity analysis. The paired bound borrows the E2 paired spread, rather than estimating E6 variability from replicated twins, so its interpretation is conditional on that assumption. The general comparison table's 0.003997 gap between GQA-2-tok and the GQA-2 *mean* is not the same-seed gain.

Using B0 s1 throughout makes the decomposition internally consistent:

$$\underbrace{0.006409}_{\text{GQA-2 final gap}} = \underbrace{0.008723}_{\text{matched-token gap}} - \underbrace{0.002314}_{\text{longer-horizon gain}},$$

$$\underbrace{0.010938}_{\text{MQA final gap}} = \underbrace{0.014460}_{\text{matched-token gap}} - \underbrace{0.003522}_{\text{longer-horizon gain}}.$$

Thus longer training recovers about 26.5% and 24.4% of the respective matched-token penalties, leaving most of the loss intact. These matched-token penalties include value-embedding and optimiser changes as well as K/V sharing. Furthermore, each run decays its learning rate relative to its own horizon: decay begins at update 840 for the token controls, 875 for full GQA-2 and about 894 for MQA. The twins share initial weights and input batches, but cease to share an optimisation trajectory once their schedules diverge. E6 therefore measures *extra tokens plus the corresponding longer schedule*, rather than the causal value of adding tokens to an otherwise identical trajectory.

4.4 Training throughput and memory

GQA-2 improves steady training throughput by 2.84%, supporting H2b's under-5% prediction; MQA improves it by 6.41%. GQA-2's roughly 4.0% FLOP reduction therefore does not translate one-for-one into tokens/s. Achieved estimated TFLOP/s falls from 372.6 to 367.8. Narrower projection matrices and work outside the FLOP model are plausible contributors, but no kernel profiler was saved to attribute the difference. MQA's 6.0% FLOP reduction produces a speedup close to the ideal inverse-FLOP ratio. Attention products still operate over all query heads, limiting the training saving available from K/V sharing.

R75 and R25 differ from full-RoPE GQA-2 by only -0.14% and -0.45% in steady throughput. This meets H2c's under-2% prediction, but shows no measured training acceleration from partial RoPE. The rotary arithmetic may be too small a share of total runtime to yield a visible gain; these timings alone do not identify why. Removing value embeddings improves B0 throughput by 2.31% despite essentially unchanged estimated FLOPs. Conversely, PM is 0.89% slower than B0 despite matching its FLOPs. These observations demonstrate why equal estimated compute and equal wall time are distinct constraints; they do not establish the exact hardware mechanism.

Table 4: Representative inference cells, using seed 1 for all architectures. Decode is aggregate generated tokens/s (255 timed tokens per request); TTFT includes prefill and first-token production. Peak allocation includes prefill.

Batch	Prompt	Model	Decode tok/s	TTFT ms	KV GiB	Peak GiB
1	128	B0	268.3	4.18	0.00586	0.286
1	128	GQA-2	257.3	4.41	0.00293	0.217
1	128	MQA	230.2	4.64	0.00146	0.181
64	1,792	B0	16,649	56.34	2.000	5.452
64	1,792	GQA-2	16,223	51.37	1.000	4.331
64	1,792	MQA	15,127	48.95	0.500	3.769
64	8,192	B0	16,206	259.21	8.250	23.033
64	8,192	GQA-2	15,282	237.08	4.125	18.592
64	8,192	MQA	16,072	225.91	2.0625	16.370

Including compilation, interim validation and checkpointing, phase throughput is 0.991M tokens/s for B0 and 1.034M for GQA-2 (+4.28%). GQA-2's 4.17% larger token target leaves duration almost unchanged (443.91 versus 444.37 seconds). All phases last 406.36–456.35 seconds, well below the cap. Peak allocation is far below H100 capacity and median loader waits are at most 0.556% of update time.

4.5 Inference: cache savings without a decode speedup

The allocated BF16 K/V cache obeys

$$M_{\text{KV}} = 2\, n_{\text{layer}}\, B\, (P + 256)\, n_{\text{kv}}\, d_{\text{head}}\, 2 \text{ bytes},$$

where the first factor accounts for keys and values, B is batch size and P is prompt length. All 168 workloads succeeded. GQA-2 halves this allocation and MQA quarters it at every shared workload, and all recorded working-set checks satisfy the registered no-expanded-K/V gate (HIa). The gate tests for a large temporary expansion under the saved benchmark procedure; it is not a proof about every possible backend implementation. At batch 64 with 1,792-token prompts, cache allocation is exactly 2/1/0.5 GiB for B0/GQA-2/MQA. At 8,192 prompt tokens it is 8.25/4.125/2.0625 GiB. These are cache ratios, not total-VRAM ratios.

At batch 64, GQA-2/B0 decode ratios are 0.954, 0.974 and 0.943 for prompts 128, 1,792 and 8,192; MQA/B0 ratios are 0.899, 0.909 and 0.992 (Figure 5). All contradict HIc's predicted speedup. At batch 1, prompt 128, GQA-2 is 4.1% slower and MQA 14.2% slower, contradicting HIb's universal under-5% prediction; GQA-2 is also 11.0% and 9.0% slower at longer prompts.

Prefill differs: batch-64 TTFT falls by 8.8%/13.1% for GQA-2/MQA at prompt 1,792 and by 8.5%/12.8% at prompt 8,192. Thus lower prefill latency coexists with slower decode. Eager

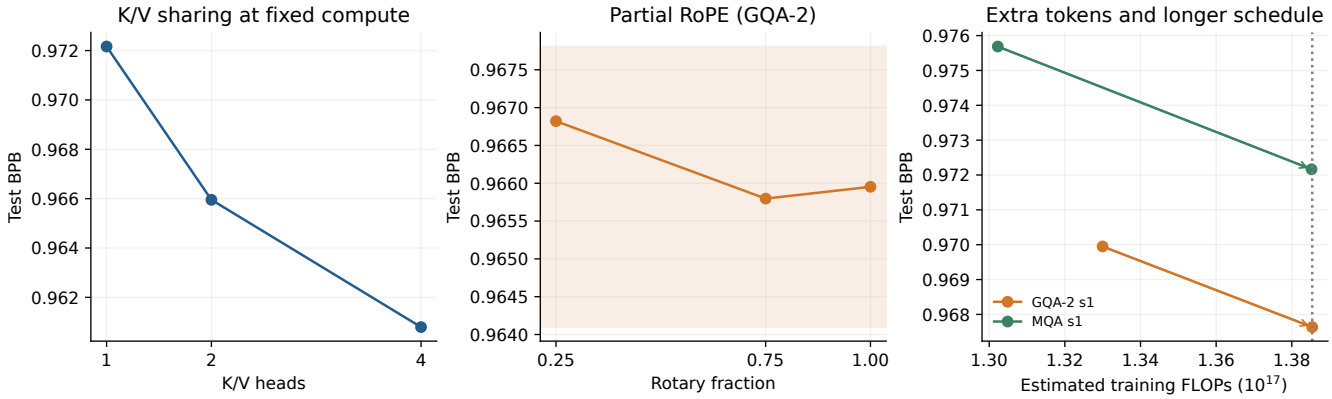


Figure 4: Left: fixed-compute K/V dose–response (MHA/GQA-2 means; MQA seed 1). Centre: rotary dose–response, with the heuristic $\pm\tau$ band around the GQA-2 mean (not an equivalence region). Right: matched-seed E6 comparisons; arrows connect token controls to their longer fixed-compute runs. The dotted line marks budget C.

execution without CUDA graphs, launch overhead, grouped-head kernel efficiency and windowed cache reads may explain the missing speedup; no profiler isolates their contributions. Two warmups and five repeats were used, but per-repeat dispersion and randomized timing order were not retained, so differences are descriptive. Every workload fits in memory; maximum feasible batch size was not tested. The 8,192-token grid measures systems performance only, beyond the training context.

4.6 Compute accounting and actual expenditure

The 14 run pipelines totalled 8,326.53 seconds (2.313 hours), or $10.25 at $4.43/hour, including training, final evaluation, inference and pipeline overhead. This is a rate-based estimate, not an invoice. The cumulative ledger, including earlier work and allowances, ends at $12.68 with zero reserved. Post-pilot and launch forecasts were $21.38 and $21.68, including $2.08 and $2.38 already spent, respectively; both allowed about $19.30 for remaining work. Most excess came from conservative overhead ($7.24 forecast versus $0.27 measured) and final-evaluation estimates ($2.81 versus $0.90; Appendix Figure 8). All training targets were completed; lower spend did not reduce the per-model scientific budget C.

5 Discussion

Arithmetic savings did not yield the best fixed-compute quality: extra tokens recovered only about one quarter of the matched-token penalties. The transferred Chinchilla coefficients overpredicted that benefit and were not fitted to this architecture or corpus. Cache savings were realised, while decode acceleration was not; this conclusion is conditional on the tested runtime. Table 5 retains both successful and failed predictions.

Deployment tradeoff. Relative to MHA's 0.960795 test BPB, GQA-2 increases BPB by 0.005158 (0.54%) and MQA by 0.011370 (1.18%); lower BPB is better. At batch 64 and a 1,792-token prompt, their allocated K/V caches fall from 2.0 GiB to 1.0 and 0.5 GiB, while peak allocated memory falls from 5.452 GiB to 4.331 and 3.769 GiB. Decode throughput at this workload falls from 16,649 to 16,223 and 15,127 tokens/s. Thus the measured tradeoff is lower memory use at a quality cost, without a decode-throughput gain on the tested eager runtime. Figure 6 places every fixed-compute architecture on

Table 5: Registered predictions and observed outcomes. "Unresolved" refers to the specified detection procedure, not evidence of equality. Failures are retained rather than reformulated after observing results.

Hypothesis	Prediction	Outcome
E1	B0 $s \in [0.001, 0.003]$	Failed: $s = 0.000656$; GQA-2 variance larger.
H2a	GQA-2 not resolved-worse	Failed: +0.005158 BPB; robust to pooled/paired checks.
H2b	GQA-2 training gain $< 5\%$	Held: +2.84% steady tokens/s.
H2c	R75 unresolved; speed difference $< 2\%$	Held: -0.000156 BPB; -0.14% throughput.
H3	PM within B0 threshold	Failed by primary rule; pooled interval unresolved.
H4a	Monotone K/V quality loss	Held: MHA $<$ GQA-2 $<$ MQA BPB; pooled checks agree.
H4b	R25 unresolved from R75/full	Held under detection rule; not equivalence.
H5a	Value-embedding removal hurts B0	Held: +0.016746 BPB; pooled check agrees.
H5b/c	Smaller GQA gap without VE; unresolved interaction	Sign agrees; +0.000507 interaction unresolved.
H6	Extra tokens save about 0.005/0.009 BPB	Overpredicted: observed 0.002314/0.003522.
HIa	No expanded-K/V-sized temporary	All 168 saved working-set checks pass.
HIb	Batch-1 decode within 5%	Failed: MQA 14.2% slower at prompt 128.
HIc	High-batch decode speedup grows then plateaus	Failed: all three batch-64 ratios below one.

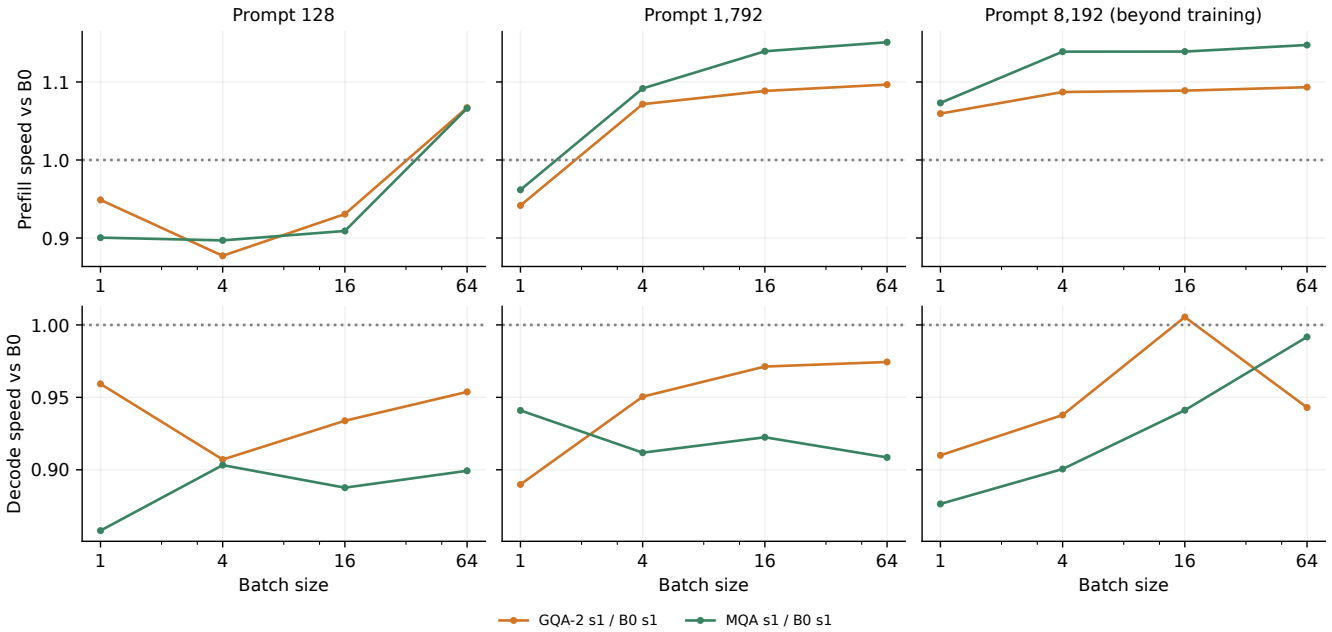


Figure 5: Inference speed relative to B0 (all seed 1) over the full benchmark grid. Values above the dotted line are faster than MHA. Top: prefill throughput. Bottom: aggregate decode throughput. The right column exceeds the training context. Absolute throughput, latency and memory for every cell are shown in Appendix Figure 7.

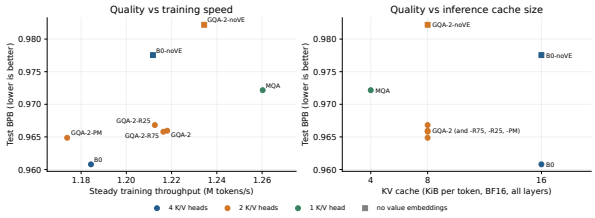


Figure 6: Fixed-compute trade-offs (three-seed means for B0 and GQA-2, one seed otherwise; token-matched controls excluded). Left: test BPB against steady training throughput. Right: test BPB against analytic KV-cache size per token. Fewer K/V heads always shrink the cache and raise BPB; they also raise training throughput, except for GQA-2-PM, which spends the saving on MLP width. Within the tested range, removing value embeddings (squares) costs more quality than reducing K/V heads.

these axes.

The appropriate model and attention configuration depend on the application, its quality requirements, context length, latency target and expected number of concurrent requests. K/V-cache demand grows with the number of live sequences and their retained tokens; when many users must be served on fixed GPU memory, a smaller model or fewer K/V heads can make higher concurrency feasible. That capacity benefit must be weighed against the measured quality loss and the absence of a decode speedup here. I did not measure maximum supported concurrency or compare different model sizes, so this is a deployment implication rather than an observed serving-capacity result.

Limitations and threats to validity.

- *Statistical power.* Only B0 and GQA-2 have three seeds; others have one. B0's threshold has two degrees of freedom and understates GQA-2 variability. Pooled-t intervals assume transferable noise and do not fix unequal variances. Seventeen contrasts have no multiplicity correction; unresolved results do not establish equivalence.
- *Compute measure.* The estimator omits embedding lookups, norms, softmax and rotary operations and charges full rather than causal-average attention windows. Equal estimated FLOPs need not imply equal operations, energy or time.
- *Confounds and tuning.* K/V sharing changes value-embedding width, gates and Muon statistics. PM matches scaling rather than total parameters and changes MLP learning-rate scaling. E5 cannot resolve attribution; E6 changes both tokens and schedule with single-seed twins. Upstream MHA hyperparameters were not tuned per variant.
- *Scope.* One depth, width, corpus and short horizon cannot establish downstream, instruction-following, long-context or larger-model quality. No data-mixture experiment or comparable GPT-2 evaluation was performed.
- *Timing and provenance.* Inference uses eager execution and fixed random prompts without randomized order or saved repeat variance. Memory is allocation, not total device usage. Held-out/tokenizer hashes agree, but training shards have filenames/sizes rather than content hashes and tokenizer inputs are checked by recipe only. The audit checks retained records, not an independent rerun.

Prioritised follow-up experiments. First, hold value-embedding capacity independent of K/V count and replicate the noVE cells. Next, repeat E6 with multiple paired seeds and a shared schedule up to the cutoff. Profile prefill/decode with randomized timing order and saved repeat distributions, test compiled/CUDA-graph execution, and extend batches until OOM. Finally, replicate at depth 12 with learning-rate

sweeps and equal-wall-time comparisons.

6 Conclusion

Under 1.3854×10^{17} estimated training FLOPs, MHA achieved the best tested mean test BPB, 0.960795. GQA-2 and MQA were worse by 0.005158 (0.54%) and 0.011370 (1.18%) BPB despite extra training, while improving steady training throughput by 2.84% and 6.41%. Partial RoPE produced no resolved quality change and no measured training speedup. GQA halved or quartered allocated K/V cache, but did not accelerate decoding in the reported high-batch comparisons. For this small-model objective, retain MHA when quality is the priority. Memory-limited serving, especially with many concurrent users or long contexts, may favour fewer K/V heads or a smaller model; the right choice depends on the application's quality, latency and concurrency requirements.

Configuration files, logs, provenance records and the commands needed to regenerate every figure and table are listed in Appendix A.

References

[ALTdJ+23] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.13245.

[APMG25] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The Polar Express: Optimal matrix sign methods and their application to the Muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.

[BBH+22] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*, 2022. arXiv:2204.06745.

[BDB+21] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, 2021. arXiv:2103.03098.

[BPC20] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[BVP+25] Federico Barbero, Alex Vitvitskyi, Christos Perivolaropoulos, Razvan Pascanu, and Petar Veličković. Round and round we go! What makes rotary positional encodings useful? In *International Conference on Learning Representations*, 2025. arXiv:2410.06205.

[DYF+25] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. Nemotron-CLIMB: CLustering-based iterative data mixture bootstrapping for language model pre-training. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025. arXiv:2504.13161.

[HBM+22] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. arXiv:2203.15556.

[HDPC20] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. arXiv:2010.04245.

[JJB+24] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

[Kar25] Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. https://github.com/karpathy/nanochat, 2025. Pinned commit 324e69c45d3606095adb6b409078647145165454.

[KMH+20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[LLL+25] Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. NorMuon: Making Muon more efficient and scalable. *arXiv preprint arXiv:2510.05491*, 2025.

[PDC+23] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, 2023. arXiv:2211.05102.

[SBZ+24] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems*, volume 37, 2024. arXiv:2407.08608.

[Sha19] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[SLP+21] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[SML+21] David R. So, Wojciech Mańke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V. Le. Primer: Searching for efficient transformers for language modeling. In *Advances in Neural Information Processing Systems*, volume 34, 2021. arXiv:2109.08668.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017. arXiv:1706.03762.

[ZWJ+24] Zhanchao Zhou, Tianyi Wu, Zhiyun Jiang, Fares Obeid, and Zhenzhong Lan. Value residual learning. *arXiv preprint arXiv:2410.17897*, 2024.

A Reproducibility and evidence

Namespace `d8-v5`, launch commit `c07aa7e`, overlays experiment code on nanochat `324e69c`. Configs, scores, logs, inference cells and provenance are in `results/d8-v5/artifacts/`; `comparison.json` holds analysis rules and sensitivity results; `results/d8-v5/audit/` holds the audit and timing CSV. All 56 final-checkpoint files match checksum markers; no consistency assertions failed. The run snapshots lacked `source/experiments/checkpoints.py`; a copy matching its recorded hash is in `results/d8-v5/export/recovered-source/`; no training effect is recorded. The corpus is excluded.

From the repository root (environment: `uv sync --frozen --extra cpu --group dev`; see `readme.md`):

- `uv run --frozen python report/verify_reproduction.py` checks, without a GPU and without writing to the repository, that the saved results match their checksums and the code that produced them;

it also regenerates the configs, comparison tables and every figure, and recomputes the headline numbers quoted in this report.

- `uv run --frozen python report/make_figures.py` regenerates `report/figures/`.
- `cd report && tectonic report.tex` (or `latexmk -pdf report.tex`) builds this PDF.

`readme.md` describes data preparation and how to rerun the 14 GPU training runs on Modal. No additional GPU runs were required.

B Additional figures

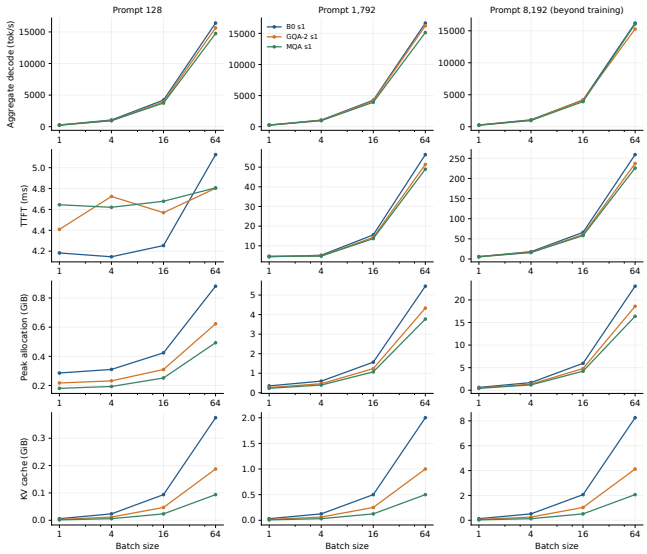


Figure 7: Inference on one H100: seed-1 MHA, GQA-2 and MQA, eager BF16, 256 generated tokens. Each prompt column has independent y limits to display all measurements. Throughput is aggregate across requests; peak allocation includes prefill. The right column exceeds the training context. No workload in this grid ran out of memory.

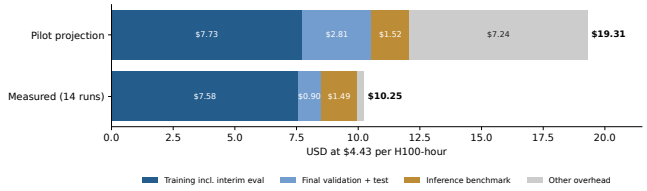


Figure 8: Pilot cost projection against the measured cost of the 14 run pipelines, at the configured $4.43 per H100-hour. The forecast mainly overestimated pipeline overhead and final evaluation; training time was predicted closely.